Radix Sort

- Radix Sort is a non-comparison-based sorting algorithm.
- It sorts numbers digit by digit, starting from the least significant digit (LSD) to the most significant digit (MSD).
- It uses another algorithm, like Counting Sort, to sort digits efficiently.

How Radix Sort Works

- 1. Find the largest number to know the number of digits.
- 2. Start sorting numbers based on the least significant digit (1s place).
- 3. Then sort based on the next digit (10s place), and continue until the most significant digit.
- 4. After sorting by all digits, the array will be completely sorted.

Example of Radix Sort

- Let's sort the numbers: [170, 45, 75, 90, 802, 24, 2, 66]

- Step 1: Sort by 1s digit → [170, 90, 802, 2, 24, 45, 75, 66]
- Step 2: Sort by 10s digit → [802, 2, 24, 45, 66, 170, 75, 90]
- Step 3: Sort by 100s digit → [2, 24, 45, 66, 75, 90, 170, 802]

C Program for Radix Sort

```
#include <stdio.h>
int getMax(int arr[], int n) {
  int max = arr[0];
  for (int i = 1; i < n; i++)
    if (arr[i] > max)
      max = arr[i];
  return max;
}
```

Advantages and Disadvantages

- ✓ Advantages:
- - Very fast for sorting integers and strings.
- - Works efficiently for large datasets.

- ✗ Disadvantages:
- - Needs additional memory for Counting Sort.
- - Only works for data that can be represented as digits (numbers or strings).

Time Complexity

- Best Case: O(nk)
- Average Case: O(nk)
- Worst Case: O(nk)

- Here, n = number of elements, k = number of digits in the largest number.
- ☞ Much faster than comparison-based algorithms like Quick Sort or Merge Sort for integers.

Summary

- Radix Sort sorts numbers digit by digit using Counting Sort as a helper.
- It works well for integers and string-based data.
- Simple concept: Sort by 1s, then 10s, then 100s place.

- Result: Fast and efficient sorting for numeric data.